Claude Platform & API by Michaël Bettan

Definition: The Claude API is a **stateless** HTTP interface. The core endpoint, `POST /v1/messages`, takes the whole conversation (system prompt, alternating `user` / `assistant` turns, tool definitions) and returns one assistant message made of typed **content blocks** plus a `stop_reason` and token `usage`. Official SDKs (Python, TypeScript, Go, Java and others) wrap the same API and add retries, streaming helpers and types.

What your code owns: the API remembers nothing between calls and never runs your tools. Your application owns **history** (what to resend, prune or compact), **tool execution** (and authorization), **branching on `stop_reason`**, **retries**, **validation** of outputs, and **cost/latency instrumentation**. Almost every exam-style defect is one of these responsibilities done wrong.

> MENTAL MODEL
> Treat every response as a **state transition**: read `stop_reason` first, then decide whether to show text, run tools, continue, compact, fall back or stop. Code that reads `content[0].text` without checking `stop_reason` is the root of most production bugs.

Workflow: 1. Build the request (model ID, `max_tokens`, system, messages, tools, optional thinking/effort and output format) → **2. Call** (sync, stream or batch: pick the transport by who is waiting and how much work there is) → **3. Branch on `stop_reason`** (`end_turn`, `tool_use`, `max_tokens`, `refusal`, and the rest) → **4. Append & loop** (append the assistant turn unchanged, add tool results, repeat until done).

1. Model family, effort and choosing a model

Tier	Profile	Typical fit	Watch out
Haiku	Fastest, lowest cost per token	High-volume classification, routing, extraction, real-time triage, sub-agent workers	Validate on your eval set before trusting it on nuanced steps
Sonnet	Balanced capability, speed and cost	**Default starting point** for most apps and agents, coding, tool use	Don't upgrade the whole pipeline when only one step fails
Opus	Most capable, slowest, highest cost	Hardest reasoning, long multi-step agent work, planning, review of critical output	Paying top-tier prices on steps a smaller tier passes

Two independent levers:

- **Model tier** sets the capability ceiling and the price per token.
- **Reasoning depth** is a per-request setting: adaptive thinking plus `output_config.effort` (`low`, `medium`, `high`, `xhigh`, `max`; the accepted set varies by model).
- Tune them separately. A smaller tier at higher effort can beat a larger tier at low effort on some tasks, and vice versa; only an eval tells you which.

Selection rule:

- Start on the balanced tier with the least reasoning that passes.
- Move **individual steps** up or down only on eval evidence; route easy vs. hard inputs to different tiers.
- **Planner → Executor tier routing (Spec-Driven pattern):** Use **Opus** (or high effort) for a read-only planning/specification pass that produces a structured spec artifact (data models, architecture, acceptance criteria), then start a **fresh context window** on **Sonnet** passing *only* the finalized spec to execute tool calls. This avoids paying Opus rates across dozens of implementation turns and keeps exploratory research noise out of the execution window.
- Treat every model or effort swap as a release: pinned ID, baseline comparison, rollback target.
- Turn thinking off or effort down on high-volume classifiers: thinking tokens are billed as output.

Sampling parameters on current models: on Claude 4.7 and later, setting `temperature`, `top_p` or `top_k` to any non-default value returns **HTTP 400**. Steer behavior with prompting, examples, structured outputs and effort instead. Even on older models where `temperature: 0` is accepted, output was **never fully deterministic**: test properties, not exact strings.

> COMMON TRAP
> "Use the best model everywhere" is still a tier decision, just the most expensive one. And "set temperature to 0 for consistency" fails twice: it 400s on the newest models and never guaranteed identical text anyway.

2. Messages API: request shape and roles

Field	Required?	What it does
`model`	Yes	Model ID. Pin a fixed ID in production (see §12).
`max_tokens`	Yes	Cap on **output** tokens for this response. Does not create context room or fix validation errors.
`messages`	Yes	Conversation as `user` / `assistant` turns; content is a string or an array of blocks.
`system`	No	**Top-level** system prompt (string or text blocks). There is no `"system"` role inside `messages`.
`tools` / `tool_choice`	No	Tool definitions and how Claude may use them (§3).
`thinking` / `output_config`	No	Adaptive thinking, `effort`, and `format` for structured outputs (§6–7).

Field	Required?	What it does
`stream`	No	Server-sent events instead of one JSON body (§5).
`stop_sequences`	No	Custom strings that end generation (`stop_reason: "stop_sequence"`).
`inference_geo`	No	First-party data residency: `"global"` (default) or `"us"` (§12).

- **Input content blocks:** `text` , `image` , `document` , `tool_result` (in user turns); prior `text` , `tool_use` , `thinking` , `redacted_thinking` (in assistant turns you echo back).
- **Response object:** `id` , `role: "assistant"` , `content[]` , `stop_reason` , `stop_sequence` , `usage` (`input_tokens` , `output_tokens` , `cache_creation_input_tokens` , `cache_read_input_tokens`).
- **Transport choice:** user waiting + short answer → sync. User waiting + long answer → streaming. Many concurrent in-app calls → async client. Bulk offline work → Message Batches.

```python
import anthropic
client = anthropic.Anthropic()  # reads ANTHROPIC_API_KEY from the environment

resp = client.messages.create(
    model=PINNED_MODEL_ID,
    max_tokens=1024,
    system="You are a support assistant for Acme. Answer only from the provided policy.",
    messages=[
        {"role": "user", "content": "Can I return an opened item?"},
    ],
)
if resp.stop_reason == "end_turn":
    answer = next(b.text for b in resp.content if b.type == "text")
```

System prompt design — the "Goldilocks zone" (prompt altitude):

Altitude	Characteristics	Failure mode
Too specific (brittle)	Treats the LLM as a deterministic state machine; hardcodes rigid branching ("if billing dispute, ask *exactly* 3 follow-up questions in this order") and tries to enumerate every edge case	Breaks on unscripted edge cases; high maintenance burden (if logic is 100% predetermined, use deterministic code, not an agent)
Goldilocks zone (heuristics)	Defines **1. Clear identity & scope**, **2. Goal-oriented tool guidance** (what tools exist and the goal, not rigid call sequences), **3. A repeatable reasoning framework** (*Understand* → *Gather context with tools* → *Act* → *Verify*), and **4. Explicit boundaries & escalation triggers** (e.g. legal, health/safety, or refunds above policy limits)	Generalizes cleanly to novel inputs while keeping behavior predictable and bounded
Too vague (underspecified)	High-level platitudes ("solve issues consistent with our brand essence; escalate if needed"); false assumption of shared context; undefined escalation boundaries	High variance across runs; model guesses company policy and either over-escalates or hallucinates rules

> COMMON TRAP
> **Prefilling** the final assistant turn to force a format is rejected with a 400 on Claude 4.6 and later. Use structured outputs (§6) instead. Also, `content[0]` is not always text: with thinking on, the first block is usually a `thinking` block. Select blocks by `type` .

3. The tool-use loop

Workflow: 1. Define tools (`name` , `description` covering what it does, when to use it and when *not* to, `input_schema`) → **2. Claude requests** (response has `tool_use` blocks, each with an `id` , and `stop_reason: "tool_use"`) → **3. Your code executes** (authorize, validate inputs, run; add a human gate before irreversible actions) → **4. Return results** (append the assistant turn unchanged, then one user turn with a `tool_result` per call).

Pairing rule	What breaks if you ignore it
Every `tool_use` gets a `tool_result` whose `tool_use_id` equals the call's `id`	400: result references an unknown ID, or a call is left unanswered
Results go in the **very next** user message; nothing in between	400 on the next request
In that user message, `tool_result` blocks come **first**; any text goes after them	Validation error, or the turn ends early
Parallel calls in one turn are all answered in **one** user message	Orphaned calls; splitting results across turns fails
Failures are returned as a `tool_result` with `is_error: true` and a short message	An empty string reads as "no data", so Claude confidently reports a false result

```python
messages.append({"role": "assistant", "content": resp.content})   # echo unchanged
results = []
for block in resp.content:
    if block.type == "tool_use":
        try:
            out = run_tool(block.name, block.input)  # your authz + validation inside
            results.append({"type": "tool_result", "tool_use_id": block.id, "content": out})
        except ToolError as e:
            results.append({"type": "tool_result", "tool_use_id": block.id,
                            "content": f"lookup failed: {e.short}", "is_error": True})
messages.append({"role": "user", "content": results})  # all results, one turn, first
```

`tool_choice`	Behavior	Notes
`{"type": "auto"}` (default)	Claude decides whether to call a tool	Add `"disable_parallel_tool_use": true` **inside** this object for at most one call per response
`{"type": "any"}`	Must call some tool	Forced use: not allowed with manual extended thinking, and some of the newest models return 400
`{"type": "tool", "name": "..."}`	Must call that tool	Same restrictions as `any`; prefer `auto` + `strict: true` or structured outputs where forcing is rejected
`{"type": "none"}`	No tool calls	Tools stay defined (keeps the cache prefix stable)

Tool design:

- Description = purpose + when to use + an **exclusion** sentence ("not for refunds").
- Small, well-named tool set; overlapping tools cause wrong picks.
- Mark truly required fields in `required`; use enums for closed sets.
- `strict: true` guarantees tool inputs match the schema.

Client vs. server tools & visual verification:

- **Client tools:** you execute them and return `tool_result` (which can include both `"type": "text"` and `"type": "image"` blocks, e.g., Playwright screenshots for closed-loop visual UI verification).
- **Server tools** (e.g. web search): Anthropic executes them inside the turn.
- A long server-tool loop can stop with `pause_turn`: send the assistant content back as-is to continue.

Agent taxonomy & scaling tool execution (Shallow ReAct vs. Deep Agents & CodeMode):

Architecture / Pattern	How it works	Trade-offs & when to use
Workflow / Hybrid RAG	LLM routes between predefined pipeline steps (rewrite query → retrieve → grade → answer)	Bounded pipelines with fixed control flow; cannot handle open-ended multi-step tasks
Shallow ReAct Loop (Direct JSON Tool Calling / Standard MCP)	Single `messages` loop: Reason → emit `tool_use` JSON (often via an **MCP Server** connected 1:1 to an **MCP Client** inside the **Host**, reducing M × N integrations to M + N) → append `tool_result` → repeat	Best for **1–5 tool iterations**. **4 scaling limits:** (1) **Upfront schema bloat** (50+ MCP tools burn ~50k tokens on turn 1), (2) **Ping-pong latency** (*K* chained calls = *K* full LLM inferences + intermediate `tool_result` bloat), (3) **JSON vs. code pretraining mismatch**, (4) **Schema limits** (JSON Schema enforces parameter types, not multi-step call order)
Deep Agent Harness	ReAct loop augmented with **(1) explicit planning tool (`TodoWrite`), (2) isolated subagents, (3) virtual filesystem** (`glob`, `grep`, paginated `read_file(offset, limit)`, `write_file`), and **(4) heuristic system prompt**	Best for **long-horizon tasks** (30+ turns). Offloads bulky intermediate state to files/storage instead of `messages`, tracks `pending` / `in_progress` / `completed` steps via `TodoWrite`, and prevents blind retry loops
Programmatic Tool Calling (Cloudflare CodeMode)	Converts MCP tool schemas into a typed **TypeScript API** (with docstrings) and exposes **one tool:** a sandboxed code runner. Model writes TypeScript that loops/chains multiple API calls; a **network-isolated V8 sandbox** routes calls via RPC bindings to MCP servers and returns `console.log()` output	Best for **multi-step tool chaining** and **large API surfaces:** collapses *K* sequential LLM round-trips into **1 inference pass** and keeps intermediate payloads out of `messages`

> COMMON TRAP
>
> "Unknown `tool_use_id`" is a **wiring** bug, not a schema or description bug. And `disable_parallel_tool_use` is not a top-level request field: it lives inside `tool_choice`.

4. stop_reason values and handling

`stop_reason`	Meaning	Correct handling
`end_turn`	Natural finish	Use the response
`tool_use`	Claude is calling one or more tools	Run them, return paired `tool_result`s, call again
`max_tokens`	Hit **your** `max_tokens` cap (or the model maximum)	Raise the cap or continue the response; never parse truncated JSON as complete

stop_reason	Meaning	Correct handling
model_context_window_exceeded	Generation filled the **context window**	Treat as truncated; prune/compact history before the next turn
stop_sequence	One of your stop_sequences fired	Read stop_sequence to see which
pause_turn	A long server-tool loop was paused	Send the assistant content back unchanged to continue
refusal	Claude declined (safety classifiers)	Returned as **HTTP 200**. Read stop_details; don't blind-retry the same request; reset or rephrase the refused turn, or retry on a fallback model where offered

Refusals in practice:

- stop_details can carry a policy category (may be null) and an explanation that isn't guaranteed stable.
- Leaving the refused turn in history leads to **continued refusals**: remove or rephrase it, or clear history.
- When streaming, the refusal arrives on the message_delta event; discard the partial turn.
- An HTTP-status-only error check misses refusals entirely.

max_tokens vs. context window:

- **Input alone** larger than the window → 400 "prompt is too long", before any generation, on every model.
- Input fits, but input + max_tokens exceeds the window → newer models accept it and stop with model_context_window_exceeded if they run out.
- The API never silently drops old turns: history management is your job.

> COMMON TRAP
> A refusal is a content decision, not a transient error: exponential backoff on the same request just repeats it. And max_tokens limits output length only; raising it does nothing for an oversized prompt or a crowded context.

5. Streaming events

Event (in order)	Carries	Handler action
message_start	Message shell, initial usage	Open a buffer for the turn
content_block_start	Block index and type (text, tool_use, thinking)	Start a per-index accumulator
content_block_delta	text_delta, input_json_delta (partial tool JSON), thinking deltas	Render text; **append** JSON fragments, don't parse yet
content_block_stop	Block complete	Now parse tool input JSON and validate
message_delta	Top-level changes: stop_reason, cumulative output usage	Branch on stop_reason (refusal, tool_use, max_tokens...)
message_stop	Message complete	**Only now** commit the assistant turn to history
ping	Keep-alive	Ignore

Why stream:

- Improves **perceived** latency (time to first token) for a waiting user.
- Recommended for long requests: very long non-streamed calls risk timeouts, and the Python SDK requires streaming above roughly 21k max_tokens.
- Does **not** reduce total tokens, cost or request count.

Interrupted streams:

- "The read loop ended" ≠ "the message is complete". Only message_stop means complete.
- On a drop, discard the partial turn and retry from the last complete turn.
- SDK helpers (e.g. a final-message accumulator) do the bookkeeping for you.

> COMMON TRAP
> Saving a half-streamed assistant turn leaves a malformed tool_use in history, and every later request fails validation, often surfacing only on retries. Parse tool input at content_block_stop; commit the turn at message_stop.

6. Structured outputs

Approach	Guarantee	Use when
System-prompt output style (e.g. YAML hierarchy, concise bullets, HTML template in system)	None: best-effort prompt guidance (shapes hierarchical reasoning and cuts prose filler)	Interactive developer tools, human-readable architectural plans, streaming Markdown/HTML
JSON outputs: output_config.format = {"type": "json_schema", "schema": {...}}	The response text parses and matches the schema (constrained decoding)	The answer itself must be machine-readable (extraction, classification, CI gates, reports)
Strict tool use: "strict": true on a tool	Tool name and inputs match input_schema	Agents whose tool calls feed real systems

```
resp = client.messages.create(
    model=PINNED_MODEL_ID, max_tokens=1024,
    messages=[{"role": "user", "content": email_text}],
    output_config={"format": {"type": "json_schema", "schema": {
        "type": "object",
        "properties": {"name": {"type": "string"},
"demo_requested": {"type": "boolean"}},
        "required": ["name", "demo_requested"],
        "additionalProperties": False}}},
)
if resp.stop_reason == "end_turn":                    # check
before parsing
    data = json.loads(next(b.text for b in resp.content if b.type
== "text"))
```

Production constraints:

- **Shape ≠ correctness.** A schema-valid object can still hold a wrong date or a hallucinated value. Keep evals and business-rule validation.
- **Still check stop_reason.** A refusal or max_tokens stop can leave output that doesn't conform.
- **First-call latency.** A new schema is compiled on first use and then cached, so the first request is slower.
- JSON outputs and strict tools can be used together in one request; both replace the old prefill trick, which newer models reject.

COMMON TRAP
When a parser breaks on "almost JSON", don't add a third paragraph of formatting instructions or a regex repair step. Move format enforcement into the API with `output_config.format` or `strict: true`.

7. Adaptive thinking

What changed: the older **extended thinking** mode made you allocate a fixed token budget. **Adaptive thinking** lets the model judge how much to reason per request; you steer depth with `effort`.

- API shape: `thinking={"type": "adaptive"}` plus `output_config={"effort": "high"}`.
- Manual `{"type": "enabled", "budget_tokens": N}` is deprecated on Claude 4.6 and returns **400 on 4.7 and later**.
- Migration: drop `budget_tokens`, switch the type to `adaptive`, control depth with `effort`.

Display and billing:

- `thinking.display`: `"summarized"` returns readable summaries; `"omitted"` returns an empty thinking field.
- You are billed for the **full** internal thinking as output tokens either way; display changes only what you see.
- Higher effort is not always better: it adds latency and cost. Match effort to task difficulty.

Rule in tool-use loops	Why
Echo the assistant message **exactly**, including `thinking` and `redacted_thinking` blocks, when you return tool results	Thinking blocks carry a cryptographic `signature`; editing, summarizing or filtering them gives a 400
Keep one thinking configuration per assistant turn	A tool-use loop is one assistant turn; change settings only between turns
Manual extended thinking allows only `tool_choice` `auto` or `none`	Forced tool use is incompatible with manual mode
To save context, prune or compact **older completed** turns	Never "save tokens" by stripping blocks from the active turn

COMMON TRAP
Changing `thinking` or `effort` between requests invalidates the messages prompt cache (and on some models the system and tool caches too). Pick a setting per workload and keep it stable.

8. Context window, compaction and prompt caching

Context strategy (Write / Select / Compress / Isolate)	How	Use when
Progressive disclosure & tool scoping (Select)	Expose lightweight ~100–200 token metadata (Skills / tool search) or task-scoped `tools[]`; load full schemas/instructions only on demand	Large tool catalogs or multi-MCP agents where 50+ eager `tools[]` schemas burn ~50k tokens on **turn 1**
Virtual filesystem offloading (Write & Select)	Write bulky intermediate outputs to files/DB (`write_file`); retrieve slices via `glob`, `grep`, and paginated `read_file(offset, limit)`	Long-horizon Deep Agents where keeping full file reads/search dumps in `messages` causes **over-retrieval**
Prune / Hard reset (Compress)	Drop stale tool results in code, or start a fresh `messages: []` array when switching tasks or after final spec approval	Tool outputs dominate history, or user switches to a new phase/unrelated task
Context editing (Compress)	Beta `context_management` edits that clear old tool uses (and thinking) server-side past a trigger	Long agent loops with bulky, one-time tool output
Compaction (Compress)	Beta `context_management` edit `compact_20260112`: older history is summarized into a compaction block (default trigger 150k input tokens; optional `pause_after_compaction` returns `stop_reason: "compaction"`)	Persistent agents that run for hours or days on a **single continuing task**
Sub-agent handoff (Isolate)	Give a bounded sub-task its own fresh `messages: []` array; return only a condensed summary or final artifact	Noisy exploration, parallel wave generation, or independent QA review
Measure	Token-counting endpoint before sending; watch `usage`	Always; production payloads are larger than test fixtures

DIAGNOSIS RULE (TURN 1 VS. TURN 5 FAILURE)
If an agent picks the wrong tool from **turn 1** after connecting multiple MCP servers, the culprit is **upfront tool-schema context pollution** (scope down `tools[]` or use progressive disclosure). If it picks the right tool on **turns 1–4** and fails from **turn 5**, the schemas are fine — **accumulated `tool_result` output** is crowding out instructions (prune, edit, compact, or offload to a virtual filesystem).

Context Rot taxonomy (3 long-context failure modes) & Multi-agent orchestration rules:

- **1. Context poisoning:** A hallucination, false assumption, or corrupted tool output enters `messages` and compounds across future turns. **Remedy:** Prune/rewind the poisoned turns or hard-reset `messages: []` (summarization/compaction often *preserves* the poisoned claim inside the summary!).
- **2. Context confusion:** Unnecessary or bulky context (full 1,500-line file reads — **over-retrieval**, irrelevant docs, or unused tool schemas) dilutes attention and steers the response. **Remedy:** Paginate reads (`offset` / `limit`), prune old `tool_result`s, or isolate in a subagent.

- **3. Context clash:** Conflicting rules or instructions appear across the prompt (e.g., system prompt rules vs. stale earlier turns or injected docs). **Remedy:** Deduplicate instructions and drop superseded turns.
- **Multi-agent context contracts (Subagent handoff, Wave fan-out & Builder–Reviewer separation):**
- **Self-contained handoff & final-message contract:** A subagent call (`messages.create`) has **zero parent history**, and its internal turns are discarded on return — the orchestrator must pass a self-contained prompt, and the subagent's system prompt must enforce returning the **raw artifact** (not a prose summary like *"Done! I made the diagram"*).
- **Wave-based fan-out (3–5 per wave) & when NOT to parallelize:** Because isolated subagents cannot see sibling agents, pass each worker a **state snapshot + unique assignment ID/filename + uniqueness constraint**, and dispatch large fan-outs in **waves of 3–5** to prevent `429` storms and orchestrator context overflow. **Never parallelize** sequentially coupled tasks (e.g., building a backend API and its frontend consumer simultaneously without a pre-agreed spec), shared mutable state/migrations, trivial <2-turn tasks, or root-cause debugging.
- **Builder–Reviewer isolation (bias mitigation):** Asking the builder to critique its own code/UI in the same `messages` thread anchors it to its prior `thinking` and rationalizations. Always run the Reviewer in a **separate API call with a fresh `messages` list** containing only the spec, rubric, and final diff/screenshot.

Prompt caching mechanics:

- Mark the end of a reusable prefix with `cache_control: {"type": "ephemeral"}`; up to **4 breakpoints** per request.
- Prefix order is `tools` → `system` → `messages`. A change at one level invalidates that level and everything after it.
- Cache hits need an **identical** prefix: static content first, dynamic content after the last breakpoint.
- Default TTL is 5 minutes (refreshed on use); `"ttl": "1h"` is optional.
- Track `cache_creation_input_tokens` vs. `cache_read_input_tokens` in `usage`.

Caching economics:

- Cache **write:** 1.25× base input (5-min TTL) or 2× (1-hour TTL).
- Cache **read:** about 0.1× base input (lower on some newer models; check pricing).
- Break-even: a 5-min entry pays off after 1 read; a 1-hour entry after 2 reads.
- Also cuts time to first token on long prefixes.
- Silent invalidators: editing tool definitions, changing `tool_choice`, toggling images, changing thinking/effort, or **swapping `system` output styles/personas mid-session** (which busts both `system` and the entire downstream `messages` cache — run separate parallel sessions per persona or place dynamic format shifts after the last `messages` breakpoint).

> COMMON TRAP
> A 0% hit rate with a timestamp, user ID or ticket text placed *before* the breakpoint. Each request writes a new entry and never reads one. A longer TTL only makes those writes cost more; move the dynamic text after the breakpoint.

9. Message Batches

- **What it is:** an asynchronous endpoint that accepts many Messages requests in one submission and processes them in the background at **50% of standard price** (input and output).
- **Limits:** capped at **256 MB total payload or 100,000 requests** per batch (whichever threshold is reached first). Most batches finish within an hour; anything not done in **24 hours** expires.
- **`custom_id`:** required, unique per request (1–64 chars, letters, digits, `_`, `-`). Results can come back **in any order**, so match on `custom_id`, never on position.
- **Lifecycle:** create → poll until `processing_status` is `ended` → fetch results (each succeeded, errored, canceled or expired). Results stay downloadable for 29 days.

Workload	Best transport	Why
Nightly classification of 40k tickets, read next morning	**Message Batches**	Half price, no rate-limit fight, latency doesn't matter
Chat UI with long answers	Streaming	A person is waiting; perceived latency
Backend fan-out of a few hundred calls per user action	Async client with bounded concurrency	Needs results in seconds
Same 5k-token instructions on every batched item	Batches + prompt caching	The discounts can be combined

> COMMON TRAP
> Splitting a list into chunks and calling `messages.create()` per item is **not** batching: it is serial synchronous traffic that hits rate limits. Smaller chunks or more backoff don't change that; the Batches API does.

10. Errors and retries

Status / type	Class	Action
400 `invalid_request_error` (bad shape, prompt too long, unsupported param, spend limit you set)	Terminal	Fix the request; retrying the same body fails identically
401 authentication · 403 permission · 404 not found	Terminal	Fix key, permissions or model ID; fail fast
402 `billing_error` · 413 `request_too_large`	Terminal	Fix billing; shrink the payload below the endpoint's size limit
429 `rate_limit_error`	Retriable	Wait `retry-after`, then backoff + jitter. A spend-cap 429 has **no** `retry-after` and keeps failing
500 `api_error` · 504 `timeout_error`	Retriable	Exponential backoff; for 504 on long requests, stream instead
529 `overloaded_error`	Retriable	Backoff with jitter; consider a fallback model in the orchestration layer
HTTP 200 + `stop_reason: "refusal"`	Not an HTTP error	Content decision: handle as in §4, don't backoff-retry

SDK defaults:

- Official SDKs retry connection errors, 408, 409, 429 and ≥500 **2 times by default** with short exponential backoff, honoring `retry-after`.

- Configure with `max_retries` (client-wide or per request); `0` disables.
- Log the `request-id` response header for support.

Reliability placement:

- **Retry:** next to the call, owned by *one* layer.
- **Circuit breaker:** at the service boundary, on aggregate error rate.
- **Fallback** (other tier, cached answer, human queue): in orchestration.
- **Budgets:** max turns / tool calls per agent run.

> **COMMON TRAP**
> Wrapping SDK calls in your own 5-attempt loop with default SDK settings gives up to 15 attempts per request against a rate limit. One layer owns retries. And a bare `except Exception: pass` retries 400s pointlessly while hiding the real error.

11. Evals and cost levers

Output shape	Grader	Catches / misses
Label, number, enum, JSON field	Exact match / code assertion	Cheap, deterministic; can't judge free text
Structured object, tool trajectory	Code checks on properties (fields, ranges, expected/forbidden tools, call counts)	Robust to wording; misses tone and nuance
Free-text answer, summary, tone	LLM judge with a rubric	Scales; must be calibrated against human labels
High-stakes or novel behavior	Human review	Gold standard; slow and costly

Eval discipline:

- Define "done" first: success criteria, failure handling, cost and latency budgets.
- Golden set = representative inputs + edge cases + adversarial cases; refresh it whenever prompt, model or retrieval changes.
- Judge rules: reasoning **before** the score, anchored score bands, preferably a different model than the one graded, agreement checked against human labels.
- Change one variable per run; read per-case diffs, not just the mean.
- Assert properties, never exact strings: output varies even when correct.

Cost lever	Effect	Number / rule
Model tier per step	Largest price driver	Start balanced; move steps on eval evidence
Effort / thinking	Thinking billed as output	Low or off for classifiers and routers
Prompt caching	Cheaper repeated prefixes, faster first token	Write 1.25× / 2×, read ≈0.1×
Message Batches	Half price for offline work	50%, ≤24 h
Context size	Every resent token is billed again	Prune, edit, compact; count tokens
Call count / fan-out	Multi-agent runs multiply tokens	Fan out only for truly independent subtasks (waves of 3–5)
Workspace spend caps (`settings/limits`)	Hard ceiling against runaway agent loops or leaked keys	Configure per-workspace USD caps in Claude Console to isolate dev/CI/subagent runs from prod
Streaming	Perceived latency only	No cost change
US-only residency (first party)	Premium	1.1× on Claude 4.6+ with `inference_geo: "us"`

> **COMMON TRAP**
> Quoting the per-call cost of a pilot. Instrument tokens, latency and errors per call from day one, multiply by production volume, size for the heavy tail and p95 rather than the median, and set the reliability floor before tuning cost.

12. Deploying: platforms, data residency and model-ID pinning

Platform	Operated by / identity	Residency control	Pick it when
Claude API (first party)	Anthropic; API keys and workspaces	`inference_geo` : `"global"` (default) or `"us"` (1.1×), Claude 4.6+ only. **No EU-only value**; older models 400 on the param	No binding cloud or EU constraint; newest features and betas first
Claude Platform on AWS	Anthropic-operated, AWS billing	Same `global` / `us` geographies	First-party features with AWS procurement
Amazon Bedrock	AWS-operated; IAM, AWS bill	Global endpoint, single-region endpoints, or geographic inference profiles (e.g. `eu.` -prefixed IDs); regional costs 10% more	Workload and compliance posture already live in AWS
Google Cloud (Vertex AI)	Google-operated; Cloud IAM and billing	Global, multi-region (`us` / `eu`) or regional (some models); 10% premium off global	Workload and compliance posture already in Google Cloud; EU residency
Microsoft Foundry	Anthropic-operated, on Azure	Varies by deployment type	Customer standardised on Azure

Model-ID pinning:

- **Claude 4.6 and later:** the dateless ID (e.g. `claude-sonnet-4-6`) *is* a fixed snapshot; updates ship under new IDs.
- **Earlier models:** dated IDs (`claude-haiku-4-5-20251001`); the short alias moves to the newest dated snapshot.
- Platform formats differ: Bedrock `anthropic.claude-…` (dated models end in `-v1:0` ; optionally `global.` / `us.` / `eu.` prefixed), Google Cloud `…@YYYYMMDD` for dated models and the API format from 4.6 on.

- Every ID has its own deprecation and retirement date.

Release discipline:

- Pin the full model ID in config, not code.
- Keep the previous pin as the rollback target.
- Promote a new model only when evals meet or beat the pinned baseline.
- Feature availability differs by platform: verify each beta or tool you depend on.

- Measure latency from the customer's region with a real payload; compare total cost (fees, egress, integration), not just token price.

> **COMMON TRAP**
> "Inference must stay in the EU" cannot be met with first-party `inference_geo` (global or US only). Use Bedrock regional/EU routing or the Google Cloud `eu` multi-region, decided at scoping because residency is pass/fail. Encryption at rest does not change where inference runs.

13. Security for API apps

Risk	Control that actually bounds it	Not enough on its own
Indirect prompt injection via fetched pages, emails, files or tool output	Treat tool/fetched content as **data**; delimit it; scan it before it enters context; isolate readers from action tools	Filtering only the user's prompt
Excessive agency (agent sends, deletes, pays)	Least privilege at the **tool registry**: remove tools the task doesn't need; read-only credentials	"Never send email" in the system prompt
Side-effecting tool misuse	Deterministic authorization in your code **before** execution (allowlist, caller identity, amount scope), failing closed; human approval for irreversible actions	An output filter that runs after the tool already executed
Forged identity ("I am an admin")	Inject verified role and tenant server-side from your auth layer	Asking the model to verify the claim
Malformed or hostile tool inputs	`strict: true` plus server-side validation; parameterised queries; fixed write paths	Trusting model-generated arguments
Leaked secrets and PII	Keys in env/secret manager, never in prompts; pass reference IDs, not raw PII; per-tenant keys or workspaces	"Don't reveal secrets" instructions
Unsafe rendering of model output	Escape or sanitize before display; never execute model HTML/JS	Assuming model output is trusted
Untrusted MCP server or programmatic tool code (`Code Mode`)	Audit 3rd-party MCP code; execute model-generated tool orchestration inside a **network-isolated sandbox (e.g. V8 isolate)** with explicit RPC bindings only; restrict CI agents to isolated branches (`claude/issue/<id>`)	Running unreviewed `npx` MCP servers or `eval()` ing generated code on the host

Compliance scoping:

- Data retention, zero-data-retention and BAA eligibility vary **by feature and by platform**: confirm each feature you plan to use.
- "No training on API data" is not the same as zero retention.
- Audit-log every tool call and decision (request, response, context, outcome) with a shared trace ID.

> **COMMON TRAP**
> "All our users are trusted employees, so injection isn't a risk." The attacker is whoever wrote the web page, email or document the agent reads. Prompt rules and bigger models are probabilistic; capability removal and pre-action authorization are not.

14. Failure modes and find the defect

Symptom / scenario	Root cause → correct fix	Tempting wrong answer
"tool_result references unknown tool_use_id"	Result ID ≠ call ID, or result not in the next user turn → echo `block.id` ; results first in the next message	Rewriting the tool description
400 after "trimming" history to save tokens with thinking on	Thinking block edited/dropped, signature mismatch → echo turns unchanged; compact older turns	Raising `max_tokens`
Validation error only on retries after a dropped stream	Half-streamed turn saved → commit at `message_stop` ; discard partial turns	Changing the tool schema
Correct tool on turns 1–4, wrong tool from turn 5	Context crowded by tool output (**context confusion**) → prune, edit, compact, or offload to virtual FS; measure tokens	Clarifying descriptions or raising `max_tokens`
Wrong tool from **turn 1** after connecting 5 MCP servers, or 25 sequential round-trips on a loop	**Upfront schema bloat** (~50k tokens) and ping-pong latency → use progressive disclosure/scoped `tools[]` and **CodeMode** (TypeScript API in isolated V8 sandbox)	Raising `max_tokens` or setting `tool_choice: "any"`
Agent repeats a hallucinated table/ID after compaction, or approves its own broken code in review	**Context poisoning** (survived compaction) and **self-confirmation bias** → hard-reset `messages: []` for poisoning; run Reviewer in a **separate `messages` array**	Running compaction again or prompting "be more critical" in the same thread
Support agent invents policies or breaks on unscripted tickets after a 40-page `if/else` rewrite	Prompt altitude outside the **Goldilocks zone** (too vague vs. brittle state machine) → define clear scope, 4-step reasoning framework, and explicit escalation triggers	Setting `temperature=0` (400s on 4.7+)
Model says "no orders found" during a DB outage	Failure returned as `""` → `tool_result` with `is_error: true`	"Never say no orders found" in the prompt
Answer silently missing; logs show HTTP 200	`stop_reason: "refusal"` or `max_tokens` not checked → branch on `stop_reason`	Retrying on HTTP status only

Symptom / scenario	Root cause → correct fix	Tempting wrong answer
Same refusal repeats every turn	Refused turn kept in history → remove/rephrase it or reset	Exponential backoff
Nightly job hits 429 every night	Sync loop over items → Message Batches + `custom_id`	Smaller chunks
Batch results attached to the wrong tickets	Matched by position → match on `custom_id`	Sorting results by timestamp
429 storms despite "retries"	Own loop stacked on SDK retries → one retry owner, honor `retry-after`, jitter	More attempts
400 after model upgrade on `temperature` / `budget_tokens`	Unsupported on 4.7+ → omit sampling params; adaptive thinking + effort	Lowering the values
0% cache hit rate	Dynamic text before the breakpoint, or tool_choice/effort/system style changing per call → static prefix first, stable settings	1-hour TTL
Parser breaks after an upstream release; can't roll back	Moving alias → pin the full ID, keep prior pin, eval-gate promotion	Adding parser retries
EU-only residency requirement on first-party API	`inference_geo` has no EU value → Bedrock/Google Cloud EU routing	`inference_geo: "us"` + encryption
Summarizer agent forwards a thread externally	Injection via fetched email + unneeded send tool → treat content as data; remove the tool	Stricter user-input regex

Find the defect: a migrated tool-use turn. This code ran on an older model. After moving to a Claude 4.7+ pinned ID it fails or misbehaves. Find at least three defects.

```
resp = client.messages.create(
    model=NEW_PINNED_ID, max_tokens=4096,
    temperature=0,                                          #
(a)
    thinking={"type": "enabled", "budget_tokens": 8000},    #
(b)
    tools=TOOLS, messages=messages,
)
messages.append({"role": "assistant",                       #
(c)
                 "content": [b for b in resp.content if b.type ≠
"thinking"]})
call = next(b for b in resp.content if b.type == "tool_use")
try:
    out = run_tool(call.name, call.input)
except Exception:
    out = ""                                                #
(d)
messages.append({"role": "user", "content": [
    {"type": "text", "text": "Here is the result:"},        #
(e)
    {"type": "tool_result", "tool_use_id": call.id, "content":
out},
]})
```

Fixes:

- **(a)** Remove `temperature`: non-default sampling values return 400 on Claude 4.7+. Steer with the prompt instead.
- **(b)** Replace the manual budget with `thinking={"type": "adaptive"}` and `output_config={"effort": "high"}`; `budget_tokens` returns 400 on 4.7+.
- **(c)** Append `resp.content` unchanged: filtering thinking blocks breaks the signature check on the next call.
- **(d)** Return failures as a `tool_result` with `is_error: true` and a short message, never `""`.
- **(e)** Put `tool_result` blocks first; text after.
- **Bonus:** check `resp.stop_reason == "tool_use"` before looking for a call, and loop over *all* `tool_use` blocks in case of parallel calls.

15. Glossary

Term	Definition
Content block	Typed unit of a message: `text`, `image`, `document`, `tool_use`, `tool_result`, `thinking`, `redacted_thinking`.
`stop_reason`	Why generation stopped: `end_turn`, `tool_use`, `max_tokens`, `stop_sequence`, `pause_turn`, `refusal`, `model_context_window_exceeded`.
`tool_use_id`	Field on a `tool_result` that must equal the `id` of the `tool_use` block it answers.
`is_error`	Flag on a `tool_result` telling Claude the tool failed, so it can retry, ask, or stop instead of trusting empty data.
`tool_choice`	`auto` (default), `any`, `tool`, `none`; holds `disable_parallel_tool_use`.
Strict tool use	`strict: true` on a tool: inputs are guaranteed to match its `input_schema`.
`output_config.format`	JSON outputs: constrains the response text to a JSON schema.
Adaptive thinking	`thinking: {type: "adaptive"}`; the model sizes its own reasoning, steered by `output_config.effort`.
Effort	Per-request reasoning depth: `low` → `max` (set varies by model). Higher = slower and costlier.
Signature	Cryptographic field on thinking blocks; any edit makes the next request fail.
`message_stop`	Final streaming event; the only safe point to commit a streamed turn.
`input_json_delta`	Streaming fragment of tool input JSON; parse only at `content_block_stop`.
Cache breakpoint	`cache_control` marker ending a reusable prefix; up to 4 per request; TTL 5m (default) or 1h.
Compaction	Server-side summarization of older context into a compaction block so long-running agents keep going.
`custom_id`	Your unique key per batched request; the only reliable way to match out-of-order results.
`inference_geo`	First-party residency parameter: `global` or `us` (Claude 4.6+).
Pinned model ID	An ID whose underlying model never changes; the unit of release and rollback.
Indirect prompt injection	Instructions hidden in content the app reads (pages, emails, tool output) rather than typed by the user.
Goldilocks zone (altitude)	System prompt sweet spot between brittle state-machine rules (too specific) and platitudes (too vague): clear scope + reasoning framework + explicit boundaries.
Context poisoning / confusion / clash	Long-context failure modes: **poisoning** (hallucination in history compounds), **confusion** (bulky/irrelevant context steers output), **clash** (conflicting rules).

Term	Definition
Deep Agent & CodeMode	**Deep Agents** combine `TodoWrite` planning, isolated subagents, and virtual-FS state offloading; **CodeMode** runs generated TypeScript against MCP tools in a V8 sandbox in 1 turn.